The Scene: 3D, Light, and Splatter

Living ink, follows depth-water-gall.pdf. Build spec for the coding agent. Supersedes that doc's relief view; the side-view panel is kept as an option; everything else in it stands.

The principle

> **The page is always a 3D scene. 2D is that scene seen straight down. Every view can switch between 2D and 3D, and you can write in either. The facts decide; the scene only shows.**

There is one renderer, not two. The flat view and the 3D view are the same scene with different cameras, so every effect (dents, beads, sunk ink, tears, splatter, light) shows in both. Nothing here changes the record: acts, facts, seals and digests are exactly as before.

0. Fix first: the height field

Bug seen in the side-view panel: a bead deposits, sinks in, and then a blob reappears on top. Cause: depth-water-gall said dried bead height = `peak`, which resurrects the dome once the cell counts as dry. A dried bead leaves a thin film and a ring, never a dome. Fix before building the scene; the 3D view uses the same height field.

- **Above the surface:** wet = `W − capacity` when W exceeds capacity (the standing bead), plus always a thin `film` = surface-deposited fixed ink (the C_SAT precipitate / ring) × `FILM_Q`. `peak` is not used for height; keep it only for trace and show-the-moments.
- **Below the surface:** `depth = (wet within capacity + fixed absorbed) / porosity(cell)`, clamped to paper thickness, monotonic. Above + below must equal the deposited volume per cell (view-side conservation check).
- **No spikes:** depth must vary smoothly with neighbors; clamp the rate on low-capacity cells and include lateral wicking in the absorbed volume.
- **Render** side-view columns with linear interpolation between cells, not per-cell bars.
- **Test:** one dwell bead over time: height above falls while depth below rises, total constant; after full dry, above-surface height is film-thin everywhere with the ring slightly higher at the edge.

1. One renderer, two cameras

- **2D** = an orthographic camera looking straight down at the page, lit. Pan and zoom as now. Because it is lit, dents and dried beads still read as depth when the light is low.
- **3D** = a free perspective camera: orbit, zoom, free fly, double-tap to fly to a point. From a person looking down at a letter to a fly between the beads.
- **A 2D/3D toggle on every ink view:** write mode, the journal, letters, the profile, the BookViewer, lessons. Switching animates the camera between the two; the page under it does not change.
- **What stays flat:** the published look PNG. It is still the record's image: it goes in the .book, the cache, thumbnails and `lookDigest`, and it is the fallback when WebGL is unavailable, on low-power mode, or with reduced motion. The scene is how the site shows ink; the flat look is what the record carries.

2. The scene

Built from a height field per cell, all from facts already defined in depth-water-gall:

```
surface height = base + cockle − dent
bead height    = W over capacity (live, wet)  or  peak (dried)
```

```
ink             = color and amount (shown absorbance after darkening and fading)
sunk            = depth
hole            = torn
```

- **Paper:** a slab with real thickness; dents and ridges on its top surface; fibers as fine surface detail (normal map) from the paper's own fields.
- **Wet beads:** glossy, slightly translucent domes with moving highlights. **Dried ink:** color on the surface; sunk ink visible at grazing angles and on cut edges. **Tears:** real holes showing the page behind.
- **Cut plane:** drag a clipping plane through the scene to see the cross-section: paper layers, sunk ink, beads above.
- **Side-view panel (kept):** the existing 2D ink panel that shows the cross-section along a line stays as its own option, beside the 3D cut plane. Same height-field data, so the bead-height fix applies to both.
- **Engine:** WebGL2 (WebGPU where available) via three.js, pinned exact. Heights and colors stream in as textures in the same 16×16 tiles the sim uses, with level of detail by distance. Live while writing: the worker's changed tiles update the scene as beads rise and sink.
- **Height field limits:** nothing overhangs, which is correct for ink on paper.

3. Light

- **Sun by the viewer's local time** (the browser clock). Low and warm at morning, high and white at noon, long gold shadows at evening. Someone reading at night sees candles while someone else sees day.
- **Candles at night:** two candles flank the page, one each side: warm point lights (about 1800 K), soft falloff, a gentle flicker (noise, non-canonical), so bead shadows shiver. Two opposite sources give each dent two soft shadows. Dusk and dawn cross-fade.
- **Move it yourself:** drag the light anywhere, any time; release to return to the clock. Works in 2D too: a low sun over the top-down page makes the grooves and droplets stand out.
- **Shadows** by height-field marching: from each surface point, step toward the light across the height field and test for occlusion. Specular highlights on wet beads.
- Light never touches facts, digests or the record.

4. Writing in 3D

If you are looking at the page, you can write on it, from any angle. The acts are identical either way; only how the pointer is turned into a page point changes.

- **Capture by ray.** Each pointer sample casts a ray from the camera through the pointer onto the paper plane (z = 0) and takes that point in page coordinates. Ink heights are far below a stroke's width, so the plane is exact enough and keeps capture simple. Pressure, timing and pen acts are captured exactly as in 2D.
- **Acts are page coordinates**, as always (the chase spec's rule). The same pointer path in page terms gives the same acts and the same facts in 2D or 3D, so a letter written at a grazing angle verifies exactly like one written flat.
- **The chase in 3D:** the same rule (the page moves only when the writer moves right, inside the zone); in 3D the camera's target slides along the line by the chase offset instead of panning a flat page.
- **Undo** restores the full camera state from before the stroke (`cam_before` now holds position, target and angle, not just the 2D offset), still one stack with the ink snapshot.
- **Minimum angle:** below a set camera elevation (default 25°) writing is paused with a hint to tilt up, because at grazing angles a pixel covers too much page for a precise stroke.

5. Show the moments

A viewer mode that faintly bands each tick's deposit so a stroke reads like tree rings: tight rings where the pen was slow, wide where it was fast, one thick band at a pause. Built from the trace data (which moment deposited each cell); available on own and open sheets. The real look and scene have no banding (see the stamping fix).

6. Splatter

Real pens throw ink: a fine spray when the nib snags a fiber, drops flung on a fast turn, a few drops on a hard landing, a fat drop from an overloaded nib. From the smallest specks to an occasional big drop. There is no randomness in the law, so the chance comes from the paper. This is part of the law (v2), not the scene.

- **Odds come from the moment.** At each act, an integer probability `P_SPLAT = f(speed, turn sharpness, carry/flow, pressure, paper tooth at the nib) × spatter`. Snag rises with speed on upstrokes over rough cells; flick with sharp turns at speed; impact with a fast pen-down and high load; overload with a dwell at high flow.
- **The paper decides.** Whether it fires: `H(paperKey, 'splat', act index) < P_SPLAT`. If it fires, the same hash stream gives the drop count, the sizes (long tail: many specks, rarely a big drop) and the offsets (ahead along the motion, farther the faster; around the point for an impact).
- **A drop is a deposit.** Each drop adds water and the current pigment at its point. Nothing else is new: the existing rules make it bead, ring, sink, feather and dry, and in the scene it stands as a real droplet before it dries.
- **Facts, not decoration.** Splatters are in the settled facts and the digest. Same strokes on the same paper give identical splatters on every regrowth; the same strokes on another person's paper give different ones, so a tracer cannot copy them. Undo removes a stroke's splatters with it. Nothing is stored.
- **Pen setting:** `spatter` 0..N on the pen (0 = never). The flex nib defaults higher than fine nibs.

7. Flex nib tips: from a point to a square

A pointed flex nib is two tines. At rest they meet in a point (the hairline). Under pressure they spread apart sideways, so the footprint grows from a point into a short, wide rectangle, not a disc. That is why copperplate shades can start and end square: press, hold a moment so the tines open, then pull down; at the end, stop, hold, lift. With a mouse or thumb, holding in one spot stands in for pressing.

- **Footprint:** for the flex nib, a rotated rectangle (slightly rounded corners) instead of a disc. Width (across) = tine spread from pressure: `w = HAIR + SPREAD[p]`. Depth (along the nib axis) stays small and nearly constant. Orientation from the pen's `angle` field: the spread axis is perpendicular to the nib axis, which follows the writing slant (55° default), so downstrokes along the slant get full width and upstrokes stay hairline.
- **Hold builds pressure (mouse/thumb):** synthetic p ramps up while the pen dwells: `p += HOLD_RAMP × dt` up to the flex maximum, then follows the existing direction/swell model as the pen moves (pressure relaxes over the stroke). At a stop before lift, p holds, so the end is square too. With a real stylus, real pressure drives it and the ramp is off.
- **Railroading (optional, real):** when the tines are wide and flow can't keep up (supply per footprint area below `RAIL_MIN`), the middle of the footprint starves and ink lands only along the two tine edges, the classic railroad-track shade. Falls out of distributing supply to the footprint's edge strips first.
- **Acts unchanged:** pressure (real or synthetic) and the pen angle are already in the acts, so square starts regrow exactly like everything else.

8. Bug: thin lines with round blobs at every turn

Seen on some pens: a thin line everywhere, then round blots wherever the pen turns or starts. Likely causes, to check in order:

- **Unbounded carry.** Deposit is time-based (`rate × dt`), and any supply that doesn't fit becomes carry. At a turn the pen slows, carry builds, then dumps as a blot. Fix: the nib holds a limited reservoir, `carry ≤ CARRY_MAX`; beyond that, supply simply isn't released (the feed can't push more ink than the tip holds). A dwell still beads, up to a limit.
- **Supply split.** Real nibs lay ink two ways: wiped off as they move (per distance) and fed while they sit (per time, saturating). Use `supply = A × distance × width + B × dt × (1 − fill)`, where `fill` is how full the cells under the nib already are. Then fast strokes aren't starved into hairlines and slow turns don't flood.
- **Disc footprint.** Any extra ink at a turn spreads as a circle because the footprint is a disc. Section 7's rectangle fixes the shape on the flex nib.
- **Synthetic p by direction.** If the mouse direction model gives near-zero p for most directions and spikes at turns, lines go thin and turns swell. Check the p values along a sample stroke.
- **Regression check.** Compare against the build before the recent fixes (the start-dot fix, the stamping fix) on the same recorded strokes, to see whether one of them introduced it.
- **Test:** a constant-speed S-curve with a mouse: width at the turns within 1.3× of the straight parts; a dwell beads to a capped size no matter how long it is held.

Serving and privacy

- Own and open sheets: the device regrows everything and builds the scene locally, real fibers included.
- Closed sheets for visitors: the render pool serves height and color tiles (`/look?mode=scene&s=...`), never the seed or paper fields. The paper's fiber detail for a closed sheet is a generic, non-canonical paper texture; only own and open sheets show the real fibers.
- One WebGL context per page. A journal with many items renders them as textured tiles in one scene, not one canvas per item. Phones get a lower tile resolution and a frame cap.

Tests

- **Same acts:** one synthetic pointer path in page coordinates, captured through a top-down camera and through an oblique 3D camera, produces byte-identical acts and facts.
- **Toggle:** switching 2D/3D mid-draft changes nothing in the draft; undo after a camera move restores both the ink and the camera.
- **Chase in 3D:** the camera target advances only on rightward motion in the zone, exactly as the 2D offset does.
- **Minimum angle:** writing below the elevation threshold is paused; above it, capture resumes.
- **Fallback:** with WebGL disabled, every view shows the flat look and writing still works in 2D.
- **Privacy:** no paper field or seed in any scene response for a closed sheet (privacy canary extended to `mode=scene`).
- **Splatter:** spatter 0 never splatters; fast flicks splatter more than slow curves; identical strokes on the same paper give identical splatters, on different paper different ones; undo removes them; total ink accounting includes drop volume.
- **Height field:** the dwell-bead test in section 0 passes before any scene work.

- **Flex tips:** hold then pull down gives a square top; stop, hold, lift gives a square end; a quick touch gives a point; upstrokes stay hairline.
- **Light:** beauty layer only: facts and digests identical under any light, time of day or candle state.

Needs Tabor's ruling

- Default view on the site: lit 2D scene (default) or the flat look.
- Whether visitors get the 3D view on closed sheets, or only on own and open sheets.
- Minimum writing angle (default 25°).
- Default spatter level, and whether it is on by default.
- Phones: lit 2D scene by default, or the flat look to save battery.
- Flex nib: how long a hold takes to open the tines fully (HOLD_RAMP), and whether railroading is on.